

Espectáculos

**Pepe Calderón, Fernando Sola, Daniel Ayala, Ana Belén
Sánchez, Inma Hernández, Margarita Cruz, Carlos Aré-
valo, David Ruiz**



Escuela Técnica Superior de
Ingeniería Informática

Índice

1. Entrevista	2
2. Diagrama de clases	4
3. Intensión	5
4. Álgebra relacional	7
5. Script SQL para crear la base de datos	10
6. Script SQL para la carga inicial de datos	13
7. Consultas	17
8. SQL avanzado	20
9. Pruebas SQL	22

Requisitos La transcripción que aparece a continuación corresponde a una entrevista realizada al gerente de un teatro para determinar los objetivos y requisitos de una aplicación web que gestione la programación anual de espectáculos y venta de entradas a dichos espectáculos.

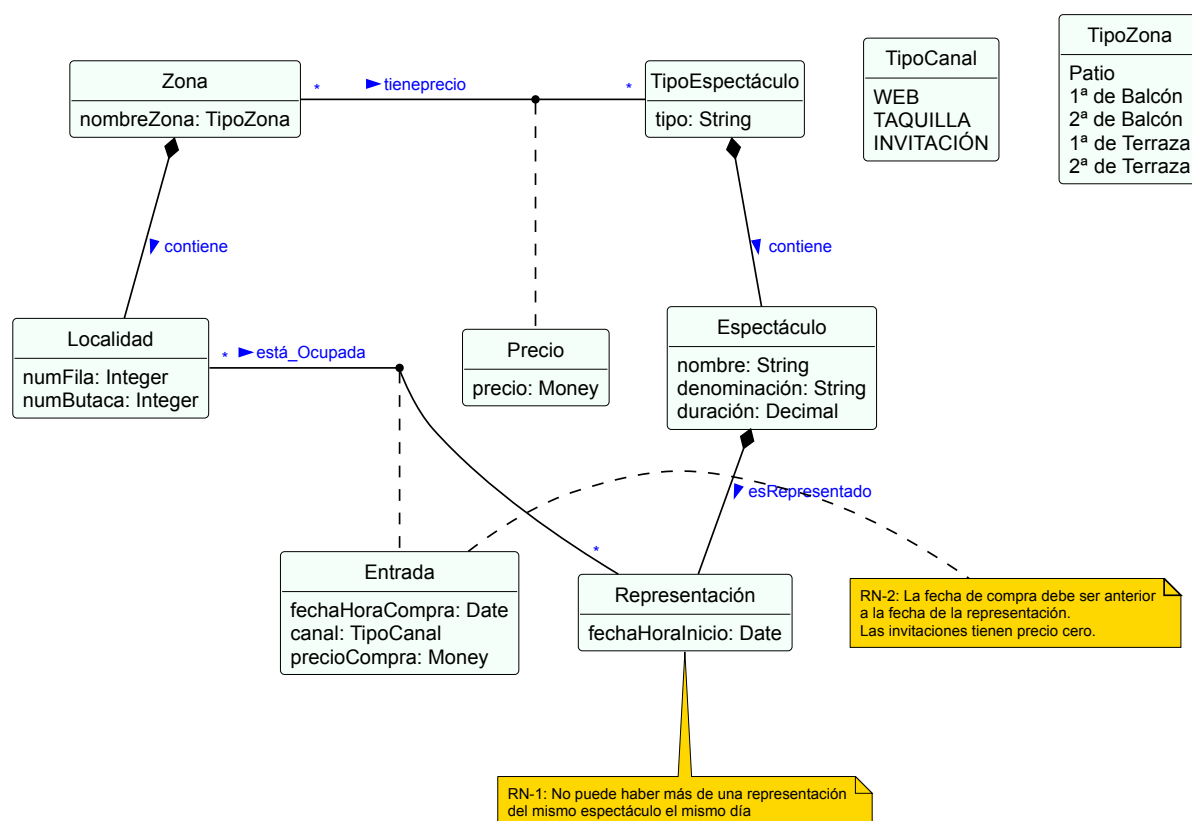
Entrevista

- Cuestión: Para empezar, hableme de los espectáculos. Me interesa conocer la información que debe gestionarse por parte de la aplicación web.
 - Respuesta: De acuerdo, los espectáculos que se programan en nuestro teatro pueden ser de distinto tipo (p.e. “Concierto”, “Opera”, ..). Cada espectáculo es de un tipo.
- Cuestión: ¿Qué información desea recoger de los espectáculos?
 - Respuesta: Básicamente, su denominación, tipo, duración y fecha y hora en las que se representan. Un espectáculo puede tener varias representaciones.
- Cuestión: ¿Puede haber más de una representación el mismo día?
 - Respuesta: Sí, hay días con más de una representación pero nunca del mismo espectáculo.
- Cuestión: ¿Algo más acerca de los espectáculos?
 - Respuesta: Se me olvidaba, también quiero guardar el precio. Cada tipo de espectáculo tiene un precio de la entrada.
- Cuestión: ¿El precio varía según tipo de espectáculo?
 - Respuesta: Sí, pero también depende de la zona del teatro donde esté situada la localidad.
- Cuestión: Acláreme esto, por favor.
 - Respuesta: El teatro está dividido en zonas. Las localidades más caras corresponden a la zona mejor situada, es decir la zona de “Patio”. Otras zonas son “1 de Balcón”, “1 de Terraza” etc. Cada zona tiene distinto precio.
- Cuestión: En definitiva ¿el precio depende del tipo de espectáculo y zona del teatro?
 - Respuesta: Así es. También quiero conocer las localidades vendidas para cada representación.
- Cuestión: ¿Hablamos de las entradas?

- Respuesta: Una entrada es una localidad para una representación. Dentro de cada zona las localidades se identifican por fila y butaca (p.e. “fila 2, butaca 3 de Patio”, “fila 2, butaca 3 de 1 de Balcón”, ...)
- Cuestión: ¿Quiere que se puedan adquirir las entradas únicamente a través de la web?
 - Respuesta: No, seguiremos vendiendo entradas en taquilla. Además algunas entradas se regalan como invitación. Necesito saber el medio o canal por el que se han adquirido las entradas ya sea web, taquilla o invitación.
- Cuestión: ¿Qué información quiere almacenar de las entradas?
 - Respuesta: La fecha y hora de compra, el canal por el que se ha adquirido, representación, fila y butaca que le corresponde y el precio de compra al que se ha adquirido ya que puede variar con el paso del tiempo.
- Cuestión: ¿Algo más?
 - Respuesta: Creo que en principio es todo. Como ya le comenté, queremos que tanto la planificación de espectáculos como la venta de entradas estén gestionados por la aplicación y se puedan consultar a través de Internet.

Modelo Conceptual

Diagrama de clases



Modelo Relacional

Intensión

```
TiposEspectaculos = { tipoEspectaculoId, tipo }
    PK(tipoEspectaculoId)
Zonas = { zonaId, nombreZona }
    PK(zonaId)
Precios = { precioId, zonaId, tipoEspectaculoId, precio }
    PK(precioId)
    FK(zonaId) / Zonas
    FK(tipoEspectaculoId) / TiposEspectaculos
Localidades = { localidadId, zonaId, numFila, numButaca }
    PK(localidadId)
    FK(zonaId) / Zonas
    AK(zonaId, numFila, numButaca)
Espectaculos = { espectaculoId, tipoEspectaculoId, nombre, denominacion, duracion }
    PK(espectaculoId)
    FK(tipoEspectaculoId) / TiposEspectaculos
Representaciones = { representacionId, espectaculoId, fechaHoraInicio }
    PK(representacionId)
    FK(espectaculoId) / Espectaculos
    AK(espectaculoId, fechaHoraInicio)
Entradas = { entradaId, representacionId, localidadId, fHoraCompra, canal, pCompra }
    PK(entradaId)
    FK(representacionId) / Representaciones
    FK(localidadId) / Localidades
    AK(representacionId, localidadId)

TiposEspectaculos = {
    (tel, "Concierto")
}

Zonas = {
    (z1, "Patio"),
    (z2, "Primera Balcón")
}
```

```
Precios = {
    (p1, z1, te1, 50),
    (p2, z2, te1, 100)
}

Localidades = {
    (l1, z1, 5, 12),
    (l2, z2, 1, 6)
}

Espectaculos = {
    (e1, te1, "AC/DC", "Concierto de ACDC", 2.5)
}

Representaciones = {
    (r1, e1, "2024-10-15 20:00"),
    (r2, e1, "2024-10-16 20:00")
}

Entradas = {
    (en1, r1, l1, "2024-10-10 18:00", "Web", 50),
    (en2, r2, l2, "2024-10-11 15:00", "Invitación", 0)
}
```

Álgebra relacional

- Precios por zona y tipo:

$$PZT \leftarrow Precios \bowtie Zonas \bowtie TiposEspectaculos$$

```
PZT = { precioId, zonaId, tipoEspectaculoId, precio, nombreZona, tipo }
```

```
PZT = {
  (p1, z1, te1, 50, "Patio", "Concierto"),
  (p2, z2, te1, 100, "Primera Balcón", "Concierto")
}
```

- Localidades por zona/tipo/representación:

$$PZTRL \leftarrow PZT \bowtie Localidades$$

```
PZTRL = { precioId, zonaId, tipoEspectaculoId, precio, nombreZona, tipo, localidadId, numFila,
numButaca }
```

```
PZTRL = {
  (p1, z1, te1, 50, "Patio", "Concierto", l1, 5, 12),
  (p2, z2, te1, 100, "Primera Balcón", "Concierto", l2, 1, 6)
}
```

- Entradas por representación:

$$numER \leftarrow \gamma_{representacionId, p_{total}(COUNT(*))} representacionId(Entradas \bowtie Representaciones)$$

```
numER = { representacionId, total }
```

```
numER = {
  (r1, 1),
  (r2, 1)
}
```

- Representaciones con espectáculos:

$$RE \leftarrow Representaciones \bowtie Espectaculos$$

```
RE = { representacionId, espectaculoId, fechaHoraInicio, tipoEspectaculoId, nombre,
denominacion, duracion }
```

```
RE = {
  (r1, e1, "2024-10-15 20:00", tel, "AC/DC", "Concierto de ACDC", 2.5),
  (r2, e1, "2024-10-16 20:00", tel, "AC/DC", "Concierto de ACDC", 2.5)
}
```

- Número de representaciones por espectáculo:

$$numRE \leftarrow \gamma_{espectaculoId, \rho_{total}(COUNT(*))} espectaculoId(Representaciones \bowtie Espectaculos)$$

```
numRE = { espectaculoId, total }

numRE = {
  (e1, 2)
}
```

- Recaudación por representación:

$$Recaudaciones \leftarrow \gamma_{representacionId, \rho_{recaudacion}(SUM(pCompra))} representacionId(Entradas)$$

```
Recaudaciones = { representacionId, recaudacion }

Recaudaciones = {
  (r1, 50),
  (r2, 0)
}
```

- Localidades vendidas en r1:

$$LocR1 \leftarrow \gamma_{representacionId, \rho_{total}(COUNT(localidadId))} representacionId(\sigma_{representacionId=r1}(Entradas))$$

```
LocR1 = { representacionId, total }

LocR1 = {
  (r1, 1)
}
```

- Entradas por invitación:

$$EntradasInvitacion \leftarrow \sigma_{canal=Invitación}(RE \bowtie Entradas)$$

4. Álgebra relacional

```
EntradasInvitacion = { representacionId, espectaculoId, fechaHoraInicio, tipoEspectaculoId,  
nombre, denominacion, duracion, entradaId, localidadId, fHoraCompra, canal, pCompra }
```

```
EntradasInvitacion = {  
  (r2, e1, "2024-10-16 20:00", te1, "AC/DC", "Concierto de ACDC", 2.5, en2, l2, "2024-10-11  
15:00", "Invitación", 0)  
}
```

Modelo Tecnológico Para crear el esquema de la base de datos en MariaDB se puede usar el siguiente script:

Script SQL para crear la base de datos

espectaculos/createDB.sql

```
--
-- Autor: David Ruiz
-- Fecha: Octubre de 2024
-- Descripción: script para crear el esquema de la BD
--

DROP DATABASE IF EXISTS EspectaculosDB;
CREATE DATABASE EspectaculosDB;
USE EspectaculosDB;

-- Crear la tabla TiposEspectáculos
CREATE OR REPLACE TABLE show_types (
    show_type_id INT AUTO_INCREMENT,
    type_name VARCHAR(255) NOT NULL,
    PRIMARY KEY (show_type_id)
);

-- Crear la tabla zonas
CREATE OR REPLACE TABLE areas (
    area_id INT AUTO_INCREMENT,
    area_name VARCHAR(255) NOT NULL,
    PRIMARY KEY (area_id)
);
```

```

-- Crear la tabla precios
CREATE OR REPLACE TABLE prices (
    price_id INT AUTO_INCREMENT,
    area_id INT,
    show_type_id INT,
    price DECIMAL(10, 2) NOT NULL,
    PRIMARY KEY (price_id),
    FOREIGN KEY (area_id) REFERENCES areas(area_id),
    FOREIGN KEY (show_type_id) REFERENCES show_types(show_type_id),
    UNIQUE(area_id, show_type_id),
    CONSTRAINT ck_prices_nonnegative CHECK (price >= 0)
);

-- Crear la tabla localidades
CREATE OR REPLACE TABLE seats (
    seat_id INT AUTO_INCREMENT,
    area_id INT,
    seat_row INT,
    seat_number INT,
    PRIMARY KEY (seat_id),
    FOREIGN KEY (area_id) REFERENCES areas(area_id),
    UNIQUE (area_id, seat_row, seat_number)
);

-- Crear la tabla Espectáculos
CREATE OR REPLACE TABLE shows (
    show_id INT AUTO_INCREMENT,
    show_type_id INT,
    name VARCHAR(255) NOT NULL,
    description VARCHAR(255),
    duration TIME,
    PRIMARY KEY (show_id),
    FOREIGN KEY (show_type_id) REFERENCES show_types(show_type_id)
);

-- Crear la tabla representaciones
CREATE OR REPLACE TABLE performances (
    performance_id INT AUTO_INCREMENT,
    show_id INT,
    start_datetime DATETIME NOT NULL,
    PRIMARY KEY (performance_id),
    FOREIGN KEY (show_id) REFERENCES shows(show_id),
    UNIQUE (show_id, start_datetime)
);

-- Crear la tabla entradas
CREATE OR REPLACE TABLE tickets (
    ticket_id INT AUTO_INCREMENT,
    performance_id INT,

```

```
seat_id INT,  
purchase_datetime DATETIME,  
channel VARCHAR(255),  
purchase_price DECIMAL(10, 2),  
PRIMARY KEY (ticket_id),  
FOREIGN KEY (performance_id) REFERENCES performances(performance_id),  
FOREIGN KEY (seat_id) REFERENCES seats(seat_id),  
UNIQUE (performance_id, seat_id),  
CONSTRAINT rn02_tickets_invitation CHECK (  
    (channel = 'Invitación' AND purchase_price = 0)  
    OR (channel IN ('Web', 'Taquilla') AND purchase_price >= 0)  
)  
);
```

Script SQL para la carga inicial de datos

Para cargar los datos de prueba se puede usar el siguiente script:

espectaculos/populateDB.sql

```
--
-- Autor: David Ruiz
-- Fecha: Septiembre 2026
-- Descripción: Procedimiento para poblar la BD de espectáculos
--

USE EspectaculosDB;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_populate()
BEGIN
    SET FOREIGN_KEY_CHECKS = 0;
    DELETE FROM tickets;
    DELETE FROM performances;
    DELETE FROM shows;
    DELETE FROM seats;
    DELETE FROM prices;
    DELETE FROM areas;
    DELETE FROM show_types;
    ALTER TABLE tickets AUTO_INCREMENT = 1;
    ALTER TABLE performances AUTO_INCREMENT = 1;
    ALTER TABLE shows AUTO_INCREMENT = 1;
```

```

ALTER TABLE seats AUTO_INCREMENT = 1;
ALTER TABLE prices AUTO_INCREMENT = 1;
ALTER TABLE areas AUTO_INCREMENT = 1;
ALTER TABLE show_types AUTO_INCREMENT = 1;
SET FOREIGN_KEY_CHECKS = 1;

INSERT INTO show_types (show_type_id, type_name) VALUES
    (1, 'Concierto')
;
INSERT INTO areas (area_id, area_name) VALUES
    (1, 'Patio'),
    (2, 'Primera Balcón')
;
INSERT INTO prices (price_id, area_id, show_type_id, price) VALUES
    (1, 1, 1, 50),
    (2, 2, 1, 100)
;
INSERT INTO seats (seat_id, area_id, seat_row, seat_number) VALUES
    (1, 1, 5, 12),
    (2, 2, 1, 6)
;
INSERT INTO shows (show_id, show_type_id, name, description, duration)
VALUES
    (1, 1, 'Concierto de ACDC', 'Festival', '02:30:00')
;
INSERT INTO performances (performance_id, show_id, start_datetime)
VALUES
    (1, 1, '2024-10-15 20:00:00'),
    (2, 1, '2024-10-16 20:00:00')
;
INSERT INTO tickets (ticket_id, performance_id, seat_id,
purchase_datetime, channel, purchase_price) VALUES
    (1, 1, 1, '2024-10-10 18:00:00', 'Web', 50),
    (2, 2, 2, '2024-10-11 15:00:00', 'Invitación', 0)
;

-- Insertar datos generados con IA:
INSERT INTO show_types (type_name) VALUES
    ('Teatro'),
    ('Danza')
;
INSERT INTO areas (area_name) VALUES
    ('Segunda Balcón'),
    ('Primera Terraza'),
    ('Segunda Terraza')
;
INSERT INTO prices (area_id, show_type_id, price) VALUES
    (3, 2, 80),
    (4, 2, 60),
    (5, 2, 40),

```

```

        (3, 3, 120),
        (4, 3, 100),
        (5, 3, 80)
    ;

    -- Insertar 3 localidades de cada zona con numero de fila y butaca
    aleatorios:
    INSERT INTO seats (area_id, seat_row, seat_number) VALUES
        (3, 5, 12),
        (3, 6, 10),
        (3, 7, 8),
        (4, 1, 6),
        (4, 2, 4),
        (4, 3, 2),
        (5, 1, 6),
        (5, 2, 4),
        (5, 3, 2)
    ;

    -- Insertar 3 espectáculos de cada tipo:
    INSERT INTO shows (show_type_id, name, description, duration) VALUES
        (2, 'El lago de los cisnes', 'Ballet', '02:00:00'),
        (2, 'El cascanueces', 'Ballet', '02:30:00'),
        (2, 'La bella durmiente', 'Ballet', '02:30:00'),
        (3, 'La casa de Bernarda Alba', 'Drama', '02:00:00'),
        (3, 'La vida es sueño', 'Drama', '02:30:00'),
        (3, 'La casa de los espíritus', 'Drama', '02:30:00')
    ;

    -- Insertar 3 representaciones de cada espectáculo:
    INSERT INTO performances (show_id, start_datetime) VALUES
        (2, '2024-10-17 20:00:00'),
        (2, '2024-10-18 20:00:00'),
        (2, '2024-10-19 20:00:00'),
        (3, '2024-10-20 20:00:00'),
        (3, '2024-10-21 20:00:00'),
        (3, '2024-10-22 20:00:00')
    ;

    -- Insertar 3 entradas de cada representación, si son de channel
    Invitación el price es 0:
    INSERT INTO tickets (performance_id, seat_id, purchase_datetime,
        channel, purchase_price) VALUES
        (3, 1, '2024-10-10 18:00:00', 'Web', 80),
        (3, 2, '2024-10-11 15:00:00', 'Invitación', 0),
        (3, 3, '2024-10-12 12:00:00', 'Taquilla', 80),
        (4, 2, '2024-10-10 18:00:00', 'Web', 60),
        (4, 3, '2024-10-11 15:00:00', 'Invitación', 0),
        (4, 4, '2024-10-12 12:00:00', 'Taquilla', 60),
        (5, 5, '2024-10-10 18:00:00', 'Web', 40),
        (5, 6, '2024-10-11 15:00:00', 'Invitación', 0),
        (5, 7, '2024-10-12 12:00:00', 'Taquilla', 40),
        (6, 8, '2024-10-10 18:00:00', 'Web', 120),

```

```
(6, 1, '2024-10-11 15:00:00', 'Invitación', 0),  
(6, 2, '2024-10-12 12:00:00', 'Taquilla', 120),  
(7, 3, '2024-10-10 18:00:00', 'Web', 100),  
(7, 4, '2024-10-11 15:00:00', 'Invitación', 0),  
(7, 5, '2024-10-12 12:00:00', 'Taquilla', 100),  
(8, 4, '2024-10-10 18:00:00', 'Web', 80),  
(8, 5, '2024-10-11 15:00:00', 'Invitación', 0),  
(8, 6, '2024-10-12 12:00:00', 'Taquilla', 80)  
;
```

```
END //
```

```
DELIMITER ;
```

```
CALL p_populate();
```

Consultas

Para crear las consultas SQL de las expresiones en Álgebra relacional se puede usar el siguiente script:

espectaculos/queries.sql

```
--
-- Autor: David Ruiz
-- Fecha: Septiembre 2026
-- Descripción: Consultas SQL correspondientes al álgebra relacional de
-- Espectáculos
--

USE EspectaculosDB;

-- PZT: precios por zona y tipo de espectáculo
CREATE OR REPLACE VIEW v_prices_areas_types AS
SELECT p.price_id, p.area_id, p.show_type_id, p.price,
       z.area_name, te.type_name
FROM prices p
JOIN areas z ON z.area_id = p.area_id
JOIN show_types te
  ON te.show_type_id = p.show_type_id;

SELECT * FROM v_prices_areas_types;

-- PZTRL: localidades por zona y tipo de espectáculo
SELECT p.*, l.seat_id, l.seat_row, l.seat_number
FROM v_prices_areas_types p
```

```

JOIN seats l ON l.area_id = p.area_id;

-- numER: Número de entradas por representación
SELECT r.performance_id, COUNT(*) AS total
FROM tickets e
JOIN performances r ON r.performance_id = e.performance_id
GROUP BY r.performance_id;

-- RE: representaciones con sus espectáculos
CREATE OR REPLACE VIEW v_performances_shows AS
SELECT r.performance_id, r.show_id, r.start_datetime,
       e.show_type_id, e.name, e.description, e.duration
FROM performances r
JOIN shows e ON e.show_id = r.show_id;

SELECT * FROM v_performances_shows;

-- numRE: Número de representaciones por espectáculo
SELECT show_id, COUNT(*) AS total
FROM v_performances_shows
GROUP BY show_id;

-- Recaudaciones: Recaudación por representación
SELECT performance_id, SUM(purchase_price) AS revenue
FROM tickets
GROUP BY performance_id;

-- LocR1: localidades vendidas en la representación 1
SELECT performance_id, COUNT(seat_id) AS total
FROM tickets
WHERE performance_id = 1
GROUP BY performance_id;

-- EntradasInvitacion: entradas por invitación
SELECT r.*, en.ticket_id, en.seat_id,
       en.purchase_datetime, en.channel, en.purchase_price
FROM v_performances_shows r
JOIN tickets en ON en.performance_id = r.performance_id
WHERE en.channel = 'Invitación';

-- Consultas adicionales

-- localidades disponibles por zona
SELECT z.area_id, l.seat_id, z.area_name, l.seat_row, l.seat_number
FROM seats l
JOIN areas z ON z.area_id = l.area_id;

-- Número de localidades vendidas para la representación 3
SELECT COUNT(*) AS total
FROM tickets

```

```
WHERE performance_id = 3;

-- entradas por invitación agrupadas por espectáculo
SELECT r.show_id, COUNT(*) AS total
FROM v_performances_shows r
JOIN tickets en ON en.performance_id = r.performance_id
WHERE en.channel = 'Invitación'
GROUP BY r.show_id;
```

SQL avanzado

Para implementar las restricciones que no pueden expresarse de forma declarativa usamos triggers. El siguiente script comprueba RN-01 y la condición temporal de RN-02; la otra condición de RN-02, relativa al precio de las invitaciones, se declara en `createDB.sql` :

espectaculos/triggers.sql

```
--
-- Autor: David Ruiz
-- Fecha: Septiembre 2026
-- Descripción: Triggers y rutinas auxiliares de EspectaculosDB
--
USE EspectaculosDB;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_check_purchase_date_rn02(
    p_representacion_id INT,
    p_fecha_compra DATETIME
)
BEGIN
    DECLARE v_start_datetime DATETIME;

    SELECT start_datetime INTO v_start_datetime
    FROM performances
    WHERE performance_id = p_representacion_id;

    IF p_fecha_compra >= v_start_datetime THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RN02: La compra debe ser anterior a la
```

```

representación';
    END IF;
END //
```

```

CREATE OR REPLACE TRIGGER t_biu_performances_rn01
BEFORE INSERT OR UPDATE ON performances
FOR EACH ROW
BEGIN
    DECLARE v_performances INT;

    SELECT COUNT(*) INTO v_performances
    FROM performances
    WHERE show_id = NEW.show_id
        AND DATE(start_datetime) = DATE(NEW.start_datetime)
        AND performance_id <> COALESCE(NEW.performance_id, -1);

    IF v_performances > 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'RN01: Un espectáculo solo puede tener una
representación al día';
    END IF;
END //
```

```

CREATE OR REPLACE TRIGGER t_biu_tickets_rn02
BEFORE INSERT OR UPDATE ON tickets
FOR EACH ROW
BEGIN
    CALL p_check_purchase_date_rn02(NEW.performance_id,
NEW.purchase_datetime);
END //
DELIMITER ;
```

Pruebas SQL

espectaculos/tests.sql

```
--
-- Autor: David Ruiz
-- Fecha: Septiembre 2026
-- Descripción: Tests negativos para EspectaculosDB
--
USE EspectaculosDB;

CREATE OR REPLACE TABLE test_results (
    test_id VARCHAR(20) PRIMARY KEY,
    test_name VARCHAR(200) NOT NULL,
    test_message VARCHAR(500) NOT NULL,
    test_status ENUM('PASS', 'FAIL', 'ERROR') NOT NULL,
    execution_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DELIMITER //
CREATE OR REPLACE PROCEDURE p_log_test(p_id VARCHAR(20), p_message
VARCHAR(500), p_status VARCHAR(10))
BEGIN
    INSERT INTO test_results VALUES (p_id, SUBSTRING_INDEX(p_message, ':',
1), p_message, p_status, DEFAULT);
END //

CREATE OR REPLACE PROCEDURE p_test_uq01_price()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('UQ01', 'UQ01:
```

```

Solo hay un precio por zona y tipo de espectáculo', 'PASS');
CALL p_populate();
INSERT INTO prices (area_id, show_type_id, price) VALUES (1, 1, 60);
CALL p_log_test('UQ01', 'ERROR: Se permitió duplicar un precio',
'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_test_rn01_performance()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('RN01', 'RN01: Un
espectáculo solo puede representarse una vez al día', 'PASS');
    CALL p_populate();
    INSERT INTO performances (show_id, start_datetime) VALUES (1,
'2024-10-15 22:00:00');
    CALL p_log_test('RN01', 'ERROR: Se permitió otra representación el
mismo día', 'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_test_rn02_purchase_date()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('RN02-A', 'RN02:
La compra debe ser anterior a la representación', 'PASS');
    CALL p_populate();
    INSERT INTO tickets (performance_id, seat_id, purchase_datetime,
channel, purchase_price) VALUES (1, 2, '2024-10-16 10:00:00', 'Web', 50);
    CALL p_log_test('RN02', 'ERROR: Se permitió comprar después de la
representación', 'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_test_rn02_invitation()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('RN02-B', 'RN02:
Las invitaciones tienen precio cero', 'PASS');
    CALL p_populate();
    INSERT INTO tickets (performance_id, seat_id, purchase_datetime,
channel, purchase_price) VALUES (1, 2, '2024-10-12 10:00:00', 'Invitación',
10);
    CALL p_log_test('RN02-B', 'ERROR: Se permitió una invitación con
precio', 'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_test_uq02_ticket()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('UQ02', 'UQ02:
Una localidad solo tiene una entrada por representación', 'PASS');
    CALL p_populate();
    INSERT INTO tickets (performance_id, seat_id, purchase_datetime,
channel, purchase_price) VALUES (1, 1, '2024-10-12 10:00:00', 'Web', 50);
    CALL p_log_test('UQ02', 'ERROR: Se permitió vender dos veces la misma
localidad', 'FAIL');

```

```

END //

CREATE OR REPLACE PROCEDURE p_run_tests()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        CALL p_log_test('POPULATE', 'ERROR: La carga inicial falló',
        'ERROR');
        SELECT * FROM test_results ORDER BY test_id;
    END;
    DELETE FROM test_results;
    CALL p_populate();
    CALL p_test_uq01_price();
    CALL p_test_rn01_performance();
    CALL p_test_rn02_purchase_date();
    CALL p_test_rn02_invitation();
    CALL p_test_uq02_ticket();
    SELECT * FROM test_results ORDER BY test_id;
    SELECT test_status, COUNT(*) total FROM test_results GROUP BY
test_status;
END //
DELIMITER ;
CALL p_run_tests();

```

i Info

Versión PDF disponible